

Especificación Técnica y Arquitectura del Sistema Marketplace Multi-Rol (Spring Boot + GCP) - V2

Este documento consolida la arquitectura, tecnologías, estándares de diseño, seguridad OWASP y la definición del catálogo de servicios HTTP segmentado según el control de acceso basado en roles (RBAC): **Administrador, Vendedor (Seller), Comprador (Buyer)** y **Público/Invitado**.

1. Stack Tecnológico Recomendado

1.1 Backend Services (API REST)

- **Spring Boot 3 + Java 21:** Microservicios con Spring MVC / WebFlux, Spring Data JPA y Spring Security.
- **Spring Security (JWT + RBAC):** Gestión de roles mediante anotaciones `@PreAuthorize("hasRole('ADMIN')")`, `hasRole('SELLER')` o `hasRole('BUYER')`.
- **Gradle / Maven:** Gestor de dependencias y automatización de builds.

1.2 Frontend & Cliente Web

- **Next.js (React) / Angular:** Implementación de SSR/ISR para optimizar SEO y rendering diferenciado por paneles de usuario (Portal del Comprador, Dashboard del Vendedor, Panel de Control del Administrador).
- **Tailwind CSS + Shadcn UI:** Interfaz moderna, limpia, responsiva y optimizada para la conversión.

1.3 Persistencia, Caché e Infraestructura (GCP)

- **Cloud SQL (PostgreSQL):** Base de datos relacional administrada para transacciones y control de inventarios multitienda.
- **Memorystore for Redis (GCP):** Almacenamiento en caché para catálogos, sesiones y reducción de carga en base de datos.
- **Cloud Run:** Despliegue de los servicios REST de Spring Boot en contenedores Docker con autoescalado.
- **Cloud Load Balancing & Cloud CDN:** Balanceador de carga HTTPS y distribución global de contenido.
- **Cloud Storage (GCS):** Almacenamiento persistente de imágenes de productos de cada vendedor.

- **Secret Manager:** Gestión segura de credenciales de base de datos, llaves API y tokens.
- **Cloud Logging & Monitoring:** Trazabilidad de logs y monitoreo de recursos en tiempo real.

2. Catálogo de Servicios y Endpoints HTTP por Rol de Usuario

2.1 Módulo de Autenticación y Gestión de Usuarios

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
POST	/api/v1/auth/register/buyer	Público	Registro de nuevos usuarios compradores.	201 Created, 400 Bad Request, 409 Conflict
POST	/api/v1/auth/register/seller	Público	Solicitud de registro para nuevos vendedores/tiendas.	201 Created, 400 Bad Request, 409 Conflict
POST	/api/v1/auth/login	Público	Autenticación global para obtener token JWT y roles asociados.	200 OK, 401 Unauthorized
PATCH	/api/v1/admin/sellers/{sellerId}/verify	ADMINISTRADOR	Aprueba o rechaza la verificación del perfil de un vendedor.	200 OK, 400 Bad Request, 403 Forbidden, 404 Not Found

2.2 Módulo de Catálogo de Productos

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
GET	/api/v1/products	Público / Todos	Listado público de productos aprobados, paginado y con filtros.	200 OK, 400 Bad Request
GET	/api/v1/products/{slug}	Público / Todos	Obtiene el detalle completo de un producto por su SLUG/ID.	200 OK, 404 Not Found
GET	/api/v1/seller/products	VENDEDOR	Consulta el catálogo propio cargado por el vendedor autenticado.	200 OK, 401 Unauthorized, 403 Forbidden
POST	/api/v1/seller/products	VENDEDOR	Crea un nuevo producto en catálogo propio (estado pendiente aprobación).	201 Created, 400 Bad Request, 403 Forbidden
PUT	/api/v1/seller/products/{id}	VENDEDOR	Actualiza la información de un producto	200 OK, 400 Bad Request, 403 Forbidden,

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
			propio.	404 Not Found
PATCH	/api/v1/admin/products/{id}/approval	ADMINISTRADOR	Modera y aprueba/rechaza un producto publicado por un vendedor.	200 OK, 400 Bad Request, 403 Forbidden, 404 Not Found

2.3 Módulo de Inventario y Stock

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
GET	/api/v1/inventory/check/{productId}	Público / Todos	Consulta la disponibilidad real de un producto.	200 OK, 404 Not Found
POST	/api/v1/seller/inventory/adjust	VENDEDOR	Ajusta las existencias físicas de los productos de su tienda.	200 OK, 400 Bad Request, 403 Forbidden
GET	/api/v1/admin/inventory/audit	ADMINISTRADOR	Auditoría global de stock e inconsistencias de inventario.	200 OK, 403 Forbidden

2.4 Módulo de Carrito, Compras y Órdenes

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
GET	/api/v1/cart	COMPRADOR / Sesión	Obtiene el carrito activo del usuario comprador.	200 OK
POST	/api/v1/cart/items	COMPRADOR / Sesión	Agrega productos con variantes al carrito de compras.	200 OK, 400 Bad Request, 409 Conflict
POST	/api/v1/orders/checkout	COMPRADOR	Convierte el carrito en una orden multi-vendedor e inicia el pago.	201 Created, 400 Bad Request, 409 Conflict
GET	/api/v1/buyer/orders	COMPRADOR	Consulta el historial de compras hechas por el cliente.	200 OK, 401 Unauthorized
GET	/api/v1/seller/orders	VENDEDOR	Consulta las órdenes asignadas a su tienda que requiere	200 OK, 403 Forbidden

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
			despachar.	
PATCH	/api/v1/seller/orders/{subOrderId}/ship	VENDEDOR	Actualiza el estado del despacho y agrega código de rastreo.	200 OK, 400 Bad Request, 403 Forbidden, 404 Not Found
GET	/api/v1/admin/orders	ADMINISTRADOR	Consulta global de todas las transacciones del marketplace.	200 OK, 403 Forbidden

2.5 Módulo de Reseñas y Moderación

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
POST	/api/v1/buyer/reviews	COMPRADOR	Crea una calificación/reseña (exclusivo para compra verificada).	201 Created, 400 Bad Request, 403 Forbidden
GET	/api/v1/reviews/product/{productId}	Público / Todos	Obtiene las opiniones verificadas del producto.	200 OK, 404 Not Found

Método	Endpoint	Rol Permitido	Descripción	Códigos HTTP
DELETE	/api/v1/admin/reviews/{reviewId}	ADMINISTRADOR	Elimina opiniones o comentarios inapropiados o fraudulentos.	200 OK, 204 No Content, 403 Forbidden, 404 Not Found

3. Mejores Técnicas de Desarrollo Web con Spring Boot

- **Arquitectura Hexagonal (Puertos y Adaptadores):** Desacoplamiento de la lógica de negocio respecto al framework y controladores REST.
- **Manejo Global de Excepciones:** Implementación de `@ControllerAdvice` con el formato estandarizado RFC 7807 (Problem Details).
- **DTOs e Inmutabilidad:** Uso de Java record y MapStruct para el filtrado estricto de campos según el rol de la consulta.
- **Concurrencia e Inventarios Multi-Vendedor:** Control de bloqueos JPA (Pessimistic Locking) al reservar productos en Cloud SQL.
- **Patrón de Resiliencia:** Integración de Resilience4j para tolerar fallos en el cálculo de envíos o consumo de pasarelas.

4. Seguridad y Reglas OWASP para Servicios Web

- **Autenticación y RBAC (OWASP API1 & API2):** Control de acceso fino mediante Spring Security y tokens JWT con claims de rol explícitos.
- **Validación de Entradas (OWASP API8):** Inyección de spring-boot-starter-validation en DTOs para evitar SQLi y XSS.
- **Protección de Datos (OWASP API3):** Comunicaciones seguras vía HTTPS/TLS 1.3 y secretos almacenados en GCP Secret Manager.
- **Limitación de Tasa por Rol (OWASP API4):** Rate limiting configurado en Cloud Armor o Bucket4j (umbrales más altos para administradores/vendedores que usuarios anónimos).
- **Cabeceras de Seguridad:** Aplicación de reglas CSP, X-Frame-Options, X-Content-Type-Options y HSTS.

5. Estándares de Diseño API RESTful

- **URIs Segmentadas por Recurso y Contexto:** Prefijos claros (/api/v1/buyer/...,

/api/v1/seller/..., /api/v1/admin/...).

- **Verbos HTTP Semánticos:** Empleo de GET, POST, PUT, PATCH y DELETE según la operación.
- **Paginación y Filtrado:** Soporte para consultas masivas con Pageable de Spring Data en catálogos y listados de órdenes.

6. Literatura y Fuentes Recomendadas

6.1 Documentación Oficial

- [Spring Boot Reference Documentation](#)
- [OWASP API Security Project](#)
- [OWASP Cheat Sheet Series](#)
- [Google Cloud Architecture Framework](#)

6.2 Libros de Referencia

- **Clean Code: A Handbook of Agile Software Craftsmanship** – *Robert C. Martin*
- **Clean Architecture: A Craftsman's Guide to Software Structure and Design** – *Robert C. Martin*
- **Effective Java (3rd Edition)** – *Joshua Bloch*
- **Domain-Driven Design: Tackling Complexity in the Heart of Software** – *Eric Evans*
- **Spring Boot in Action** – *Craig Walls*